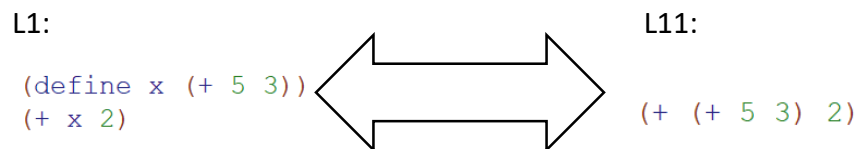


Question 1: Language variations

Q1.1 Let us define the L11 language as L1 excluding the special form 'define'. Is there a program in L1 which cannot be transformed to an equivalent program in L11? Explain or give a contradictory example.

Answer: No, because there are no recursive functions in L1, so any var reference can be replaced by its defined value expression. For example:



Q1.2 Let us define the L21 language as L2 excluding the special form 'define'. Is there a program in L2 which cannot be transformed to an equivalent program in L21? Explain or give a contradictory example.

Answer: Yes, since L2 supports user procedures (may contain recursion calls), var reference cannot be replaced by their defined value. For example:

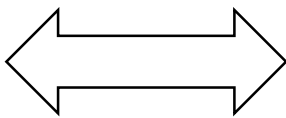
L2:

```
(define fact
  (lambda (n)
    (if (= n 0)
        1
        (* n (fact (- n 1))))))
```

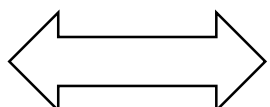
This recursive function cannot be directly transformed into L21 without define.

Q1.3 Let us define the L22 language as L2, where procedures (lambda) can only have one parameter and a body with one expression. Is there a program in L2 which cannot be transformed to an equivalent program in L22? Explain or give a contradictory example.

Answer: No, every function that have more than one parameter can be transform to a function with one parameter that have in its body another function. Like:

L2: `(lambda (x y) (+ x y))`  L22: `(lambda (x) (lambda (y) (+ x y)))`

And if it have more than one expression in its body we can transform to a function that has one expression in the body by taking the last expression as a return value. And according to a theory we take at class two functions are equivalence if and only if they take the same number of vars with the same domain and return the same value when they take the same vars. For example:

L2: `(lambda (x) (+ x 2) (+ x 3))`  L22: `(lambda (x) (+ x 3))`

Can be transform to function which has one parameter and one expression in the body.

Note: L2 didn't have side-effect (except define)

Q1.4 Let us define the L23 language as L2, where procedures (lambda) are first-order, i.e. cannot get functions as arguments. Is there a program in L2 which cannot be transformed to an equivalent program in L23? Explain or give a contradictory example.

Answer: Yes, since L23 restricts higher-order functions. We cannot use function that has an function as an argument. For example:

L2: `(define apply-two
 (lambda (f x)
 (f (f x))))`

This function takes another function f and applies it twice to x. In L23, since functions cannot accept other functions as arguments, this pattern is not expressible.

Q2.1a: new primitive

Answer: add to prim-op: dict? | get | dict

Q2.2a: special

Answer: add "Dict(<symbol> <cexp>*)/ dictExp (key: symbol , val:Cexb)"

And also we add to the syntax ast:

"export type DictExp = { tag: "DictExp"; map: DictMap[]; } "

Q2.4:

2.4.1:

- 2.1 לא ישתנה כי אנחנו מגדירים כ prime-op אז בכל מקרה יחשב הערך מראש
2.2 כן צריך לשנות כי בבנאי אנחנו מחשבים את כל הערכים ישר אז צריך לשנות כך שלא לחשב את הערכים עד שנצרך אותם
2.3 לא ישתנה כי כמו 2.1 מגדירים את הערכים כ prime-op

2.4.2: מודל הסביבה

לא, מודל הסביבה אינו משנה דבר שקשור למילונים באף אחת מהסעיפים (2.1,2.2,2.3)

2.4.3: quoting

2.1 ו 2.3 ה DICTONARY ממומש כפונקציה ולכן parsed as appExp
ואז בכתיבה בלי נקודה ((dict (a 1) (b 2)) הפארסיר ילך לחפש את הפונקציות a עם פרמטר 1 ו b עם פרמטר 2 ולא יצליח בזה ויחזיר שגיאה.

2.4.4:

- א
כן יש , למשל: (dict (x (- 5 2))) הביטוי לא יתקבל ב 2.1 ו 2.3 אך זה יתקבל ב 2.2 ,
מכיון ש 2.2 special form הפארסר לא מקבל הארגומנט ישר אז APP אלא קודם evaluate ושומר התוצאה. בניגוד ל 2.1 ו 2.3 ששומר ישר
ב
לא השדות שצריכות חישוב (evaluate) ימרו ל exp literal ,
אך שאר השדות כמו מספרים או מחרוזת אותו דבר.

2.4.5: we would prefer the special form implementation

יתרונות: ✓

- מאפשר כתיבה קלה, ברורה ופשוטה ל $(b\ 2)(a\ 5)$ dict: Dict בלי ניקוד

- תומך בביטויים מורכבים כפי שהראיתי למעלה

- תומך בבדיקות מוקדמות של שגיאות (למשל פורמט לא תקין)

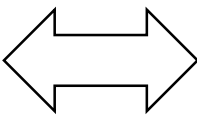
חסרונות: ✗

- צריך לשנות בפרסר Interrupter כפי שעשינו בקוד
- אינו פונקציה רגילה ואינו אופרטור פרימיטיבי, ולכן לא ניתן לשמור אותו במשתנה, להעביר אותו כארגומנט לפונקציה אחרת, או להחזיר אותו מפונקציה. בנוסף, קשה יותר לשלב אותו כחלק מביטויים מורכבים או להשתמש בו במסגרת תכנות פונקציונלי.

Q4:

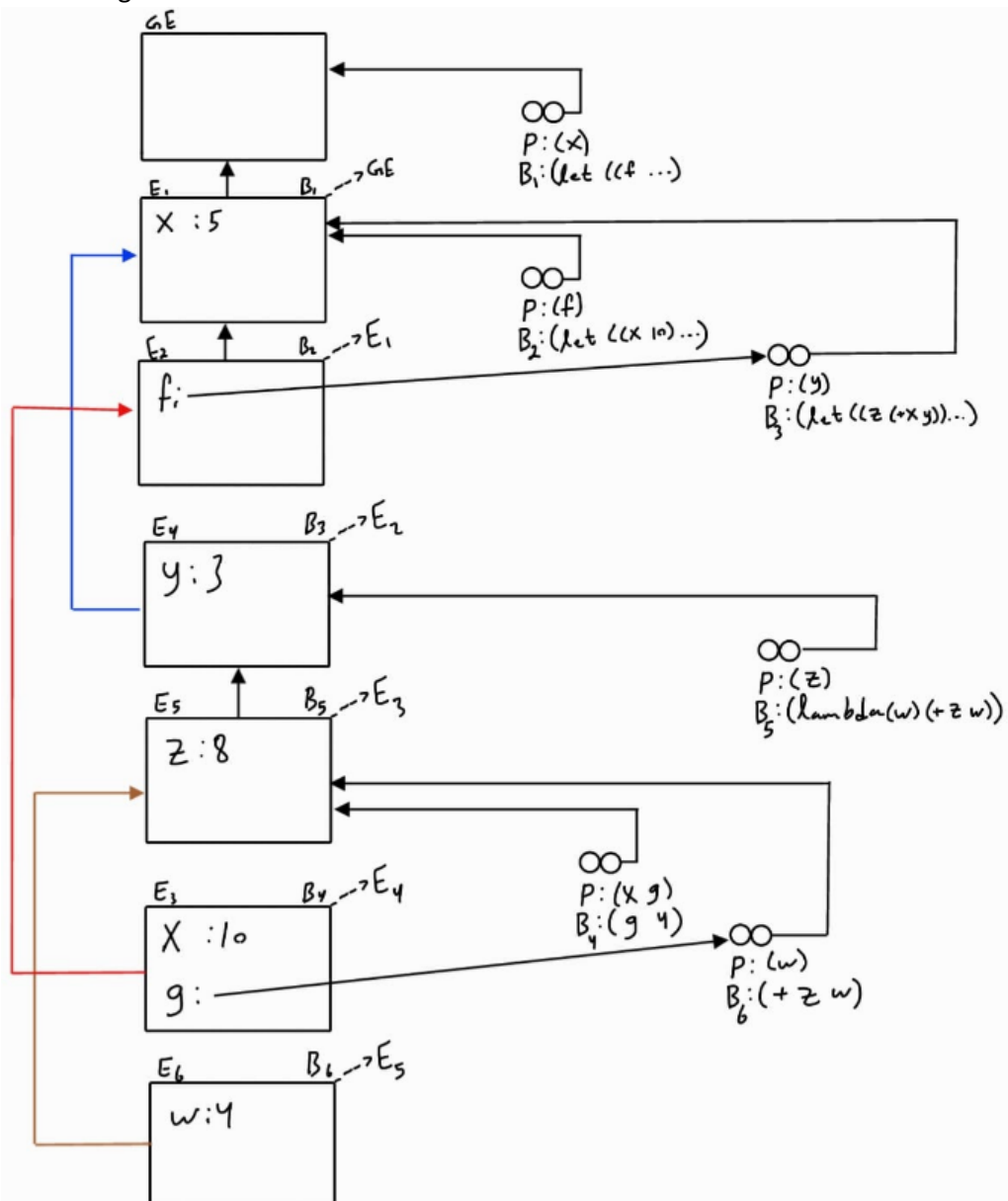
Q4a: We can convert the code we get to another form according to practice lesson (we can convert lambda to let and let to lambda):

```
(let* ((x 5)
      (f (lambda (y)
            (let ((z (+ x y)))
              (lambda (w)
                (+ z w))))))
  (let ((x 10)
        (g (f 3)))
    (g 4)))
```



```
(let ((x 5))
  (let ((f
        (lambda (y)
          (let ((z (+ x y)))
            (lambda (w)
              (+ z w))))))
    (let ((x 10)
          (g (f 3)))
      (g 4))))
```

So the diagram is:



Q4b: According to diagram we get function that take to parameters x and h while $x=2$ and $h = \lambda(y)$, and then in the body of the function we get another $f = h(x=3)$ and we apply $f(2)$ so $y = 2$. Then the code that suitable to the diagram is:

```
(let ((x 2)
      (h (lambda (x)
            (lambda (y)
              (* x y))))))
  (let ((f (h 3)))
    (f 2)))
```